To start a component in an app, we can use Explicit Intent. For example, the following code will make the activity start a subordinate service:

```
Intent int2 = new Intent(Activity1.this,
Activity2.class);
startActivity(int2);
```

Services: A service is an application component that is used to perform operations of long duration in the background and no user interaction is required. As it is running in the background, no front-end operations will affect the execution of the service. We need a service to handle all background tasks like handling network transactions, performing I/O files or playing music.

A service can take the following two forms.

1) *Started:* A Started service can run in the background indefinitely and stop itself once the operation is done. A service starts its execution when an application component called the *startService()* method is made. Destroying the component that started the service will not affect how a service running in the background is executed.
2) *Bound:* Bound services allow interaction between services; they get results, send requests and carry out interprocess communications (IPC). When an application component calls the method *bindService()*, the service will be bound with that component.

A Bound service stops once the application component bound to it is destroyed. The service cannot be stopped until all components attached to the service unbind the service.

Like activities, services also have the following important methods:

- *onStartCommand()*: When any component requests the service to be started, the system will call this method.
- *onBind()*: When any component wants to bind the service, the system will call this method.
- *onUnbind()*: This method is called once all the components unbind the service.
- *onRebind()*: This method is called when any new components try to bind the service, after the service has been unbound by all its components.

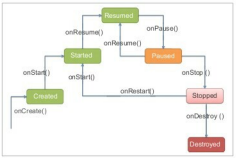

Figure 10: Activity life cycle

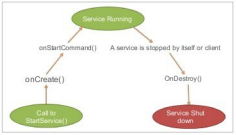

Figure 11: Started services life cycle

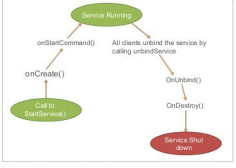

Figure 12: Bound services life cycle

- *onCreate()*: This method is called when a service is requested by any component.
- *onDestroy()*: This method is called when a service no longer exists, and it is used to clean up the resources.

Figure 11 explains the life cycle of Started services during app execution.

Figure 12 explains the life cycle of Bound services during app execution.

Content providers: A content provider component passes the data from one app to another if requested. All requests of this type are handled by the *ContentResolver* class. This is a central repository that stores the content in one place and allows different applications to access it. It is the same as a database, wherein the manipulation of content is allowed. Usually, the content is stored in the SQLite database.

Shown below is the form of the URI which we should specify in the query string to query a content provider:

```
<prefix>://<authority>/<data_
type>/<id>
```

Given below are the methods to implement the *ContentProvider* class.

- *onCreate()*: This method is called once the content provider is started.
- *query()*: This returns the result of the query.
- *insert()*: Adds a new row into the content provider.
- *delete()*: Deletes an existing row from the content provider.
- *update()*: Updates an existing row from the content provider.
- *getType()*: Returns the MIME type of the data at the given URI. END

References

[1] *http://www.javatpoint.com/android-life-cycle-of-activity*
[2] *http://www.tutorialspoint.com/android/*
[3] *https://developer.android.com/guide/topics/*
[4] *http://www.codeproject.com/Articles/628894/Learn-How-to-Develop-Android-Application*

By: Palak Shah

The author is an associate consultant in Capgemini and loves to explore new technologies. She is an avid reader and writer of technology related articles. She can be reached at *palak311@gmail.com*.